

Brak Platform Learning Compendium

Airon Brak Sovereign AI Infrastructure Curriculum

Phases 1–4 + Transformer Deep Dive

A reference guide covering everything learned so far. Each phase builds on the previous one, from physical hardware through networking, OS security, and container orchestration.

Table of Contents

Phase 1: The Physical and Hardware Foundation

Topic 1: Bare-Metal Server Architecture

Topic 2: GPU Hardware — H200, B200, B300

Topic 3: NVLink and Intra-Node GPU Communication

Topic 4: BlueField-3 DPU / SuperNIC

Phase 2: Networking and Cluster Fabric

Topic 5: Ethernet vs InfiniBand for AI Clusters

Topic 6: RDMA Fundamentals

Topic 7: NVIDIA Spectrum-X and DOCA

Phase 3: Operating System and Boot Security

Topic 8: UEFI, Secure Boot, and Measured Boot

Topic 9: TPM 2.0 and Attestation

Topic 10: Immutable OS Images

Topic 11: Hardened Ubuntu and CIS Benchmarks

Phase 4: Containers, Kubernetes, and Kata

Topic 12: Container Fundamentals

Topic 13: Kata Containers Deep Dive

Topic 14: Kubernetes Core Concepts

Topic 15: Cilium and eBPF Networking

Topic 16: Helm, Flux, and GitOps (in progress)

Deep Dive: How Transformer Inference Works

Attention Mechanism

Prefill, KV-Cache, and Decode

KV-Cache Sizing

Full Inference Flow

Phase 1: The Physical and Hardware Foundation

Topic 1: Bare-Metal Server Architecture

The key mental model

A regular server is built around the CPU — everything flows through it. A GPU server flips this: the GPUs are the real workers and the entire physical design exists to feed them as fast as possible. The CPU is just a manager — loading data, launching GPU kernels, orchestrating.

What's inside an AI server

A 4U–8U rackmount chassis containing:

- **1–2 CPU sockets** — typically AMD EPYC. EPYC wins over Intel Xeon for GPU servers because it provides more PCIe lanes (128 per socket), meaning more bandwidth highways to GPUs.
- **8 GPUs** — the standard NVIDIA reference configuration (DGX/HGX platforms), each with its own heatsink or liquid cooling.
- **1–2 TB DDR5 system RAM** — mostly for staging data before it's pushed to GPUs.
- **NVMe SSDs** — for the OS and model weight storage.

PCIe topology

PCIe (Peripheral Component Interconnect Express) is the physical bus connecting components inside the server. Each GPU needs 16 PCIe lanes. With 128 lanes per CPU socket and 8 GPUs, the physical layout of which GPU connects to which socket matters for latency.

Key insight: GPU-to-GPU communication should avoid PCIe when possible. PCIe bandwidth is ~64 GB/s per x16 slot. NVLink provides ~900 GB/s — a 14x difference.

BMC (Baseboard Management Controller)

A tiny, always-on computer embedded on the motherboard, accessed via IPMI. Provides remote power on/off, console access even when OS is dead, temperature/fan/voltage monitoring, and firmware updates.

The BMC sits on a **separate, isolated management VLAN** because it has god-level access to the server. If an attacker compromises the BMC, they own the machine entirely. This network isolation is a security fundamental in Bruk's design.

Topic 2: GPU Hardware — H200, B200, B300

HBM (High Bandwidth Memory)

GPUs have their own memory (HBM) sitting directly on the chip package via a silicon interposer — physically millimeters from the compute cores, not inches away across a circuit board like system RAM.

	DDR5 System RAM	HBM3e (H200)	HBM3e (B200/B300)
Bandwidth	~90 GB/s per channel	~4.8 TB/s	~8 TB/s
Capacity	1–2 TB total	141 GB per GPU	192–288 GB per GPU

Memory-bandwidth bound vs. memory-capacity bound

- **Memory-bandwidth bound** — the GPU is waiting for data to be read from memory faster than HBM can deliver it. During token generation, the GPU reads the entire model's weights for every single token but does little math per weight. The bottleneck is the speed of the pipe, not the size of the tank.
- **Memory-capacity bound** — the model simply doesn't fit in the available VRAM. A size problem solved by adding more GPUs.

LLM token generation is memory-bandwidth bound. HBM bandwidth directly determines token generation speed.

Tensor cores vs. CUDA cores

- **Tensor cores** — specialized circuits that do matrix multiply-accumulate on small matrices (4×4 or 8×8) in a single clock cycle. Handle the heavy lifting for transformer layers.
- **CUDA cores** — general-purpose cores that handle everything else: activation functions (ReLU, SiLU), softmax, layer normalization, sampling, data reshaping.

The three GPUs

GPU	VRAM	Key significance
H200	141 GB HBM3e	Current gen. Llama 70B in FP16 (~140 GB) barely fits on one card.
B200	192 GB HBM3e	Next gen (Blackwell). ~2x inference performance of H200.
B300	288 GB HBM3e	405B in FP8 (~405 GB) fits on just 2 cards instead of 3–4 H200s.

VRAM per GPU determines how many GPUs you need, which determines how much inter-GPU communication is required, which determines how important NVLink and RDMA are. Everything connects.

Topic 3: NVLink and Intra-Node GPU Communication

What NVLink is

A direct physical interconnect between GPUs — dedicated wires on the baseboard that bypass PCIe entirely. NVLink 4th gen (Hopper/H200) provides 900 GB/s bidirectional bandwidth across all 8 GPUs.

NVSwitch

A crossbar switch that sits between all 8 GPUs. Without it, you'd need 28 separate point-to-point connections for 8 GPUs. With NVSwitch, any GPU can talk to any other GPU at full bandwidth simultaneously. All 8 GPUs effectively share a single flat memory pool.

Important scope precision: NVSwitch is **intra-node** (inside one server), NOT intra-rack.

Why it matters for inference

When a model is sharded across multiple GPUs (tensor parallelism), each GPU holds a slice of each layer. After every layer computation, GPUs must exchange partial results before the next layer can begin. This synchronization happens for every token generated.

- Over PCIe at 64 GB/s → GPUs sit idle waiting → high latency per token
- Over NVLink at 900 GB/s → exchange is nearly instant → GPUs stay busy

The critical boundary

- **Intra-node (inside one server)** = NVLink — fast
- **Inter-node (between servers)** = Network RDMA — slower but still fast

Topic 4: BlueField-3 DPU / SuperNIC

The problem

In a GPU server, the CPU is already busy orchestrating GPU work. Piling networking tasks onto it steals cycles from the real job.

What a DPU is

A small computer on a network card. The BlueField-3 has its own ARM cores, its own memory, and its own OS. It handles:

- **RDMA** — direct memory transfers without CPU involvement
- **Encryption** — line-rate hardware encryption, no CPU overhead
- **Network isolation** — tenant separation and firewall rules in hardware
- **Storage offload** — NVMe-over-Fabrics if needed

The DPU enforces network policy in hardware. Even if the host OS is compromised, the DPU still enforces isolation. It's a separate trust boundary.

Phase 1 — The big picture

Inside a server, the data flow for an inference request: Request hits the network switch → arrives at the DPU (decrypts, checks policies) → CPU receives it (determines which GPUs handle it) → GPUs run the model layer by layer exchanging results over NVLink → output tokens flow back out the reverse path. Each layer protects and feeds the next one.

Phase 2: Networking and Cluster Fabric

Topic 5: Ethernet vs InfiniBand for AI Clusters

	InfiniBand	Ethernet
Latency	~1 μ s	~2–5 μ s (historically)
RDMA	Native, built-in	Requires RoCEv2 (add-on)
Ecosystem	Specialized, NVIDIA-dominated	Universal, multi-vendor
Cost	Expensive, fewer vendors	Competitive, many vendors
Talent pool	Small, niche expertise	Huge, everyone knows Ethernet

Why Bruk chose Ethernet

This is a **sovereignty decision**. "Sovereign" doesn't just mean data stays in the country — it means you control every layer of the stack. InfiniBand means NVIDIA switches, firmware, management tools, and NVIDIA-trained engineers. Ethernet gives you multi-vendor options, massive talent pool, standard tooling, no lock-in. InfiniBand's performance edge has shrunk — Spectrum-X with RoCEv2 gets close enough.

RoCEv2 (RDMA over Converged Ethernet)

Pronounced "rocky v2." Layers RDMA protocol on top of standard Ethernet. The catch: RDMA doesn't handle packet loss gracefully, so Ethernet must be configured as **lossless**.

Lossless Ethernet mechanisms

- **PFC (Priority Flow Control)** — the emergency brake. Tells upstream switch "stop sending, my buffer is almost full." Can cause head-of-line blocking.
- **ECN (Explicit Congestion Notification)** — the early warning system. A 2-bit field in the IP packet header. Switches flip ECN bits to "congestion experienced" when buffers start filling. Receiver notifies sender, who slows down. Proactive, not reactive. ECN is preferred; PFC is the emergency fallback.

Topic 6: RDMA Fundamentals

Normal networking vs. RDMA

Normal networking: Application → kernel copies to buffer → NIC sends → remote NIC → kernel copies to buffer → application. Multiple copies, CPU involved at every step.

RDMA: Application tells NIC "write this data directly into memory address X on Server B." The NIC does it. Done. This is **zero-copy, kernel-bypass** networking.

Why RDMA matters for Bruk

In disaggregated inference, after prefill finishes, the KV-cache must transfer from the prefill server to the decode server. This cache can be gigabytes. RDMA lets the prefill server write it directly into the decode server's GPU memory — almost as if it were local memory.

The three RDMA operations

Operation	Description	Use case
RDMA Write	Push data into remote memory. Remote CPU never knows.	KV-cache transfer

RDMA Read	Pull data from remote memory. Remote CPU uninvolved.	Less common
RDMA Send/Receive	Both sides coordinate.	Control messages

Topic 7: NVIDIA Spectrum-X and DOCA

What Spectrum-X is

NVIDIA's complete Ethernet networking platform optimized for AI. Three components:

1. **Spectrum-4 switches** — physical network switches in the rack
2. **BlueField-3 DPUs/SuperNICs** — on each server
3. **Software stack** — adaptive routing, congestion control, telemetry

Adaptive routing

Monitors congestion across all paths in real time and redirects packets along less-congested routes, **per-packet**. Critical because AI traffic is **bursty** — after every layer computation, all GPUs blast data at the same microsecond.

DOCA (Data Center Infrastructure on a Chip Architecture)

NVIDIA's SDK for programming BlueField DPUs. Lets Bruk engineers write custom networking and security logic that runs on the DPU, not the host CPU. Security policies enforced in DPU hardware — the host OS never gets a vote.

Phase 2 — The big picture

Inside a server: NVLink (~900 GB/s). Between servers: RDMA on Spectrum-X Ethernet. DPU handles network traffic without burdening CPU. Lossless Ethernet (PFC + ECN) prevents drops. Adaptive routing handles bursty AI traffic. Every layer keeps data flowing to GPUs as fast as possible.

Phase 3: Operating System and Boot Security

Topic 8: UEFI, Secure Boot, and Measured Boot

The boot chain

UEFI firmware → Shim (signed by Microsoft CA) → GRUB → Linux kernel → initrd

Secure Boot — prevention

Uses **digital signatures** (not hashes) to verify each boot stage. UEFI firmware has a database of trusted signing keys baked in. An attacker who replaces the kernel can't sign it with the correct private key. Key distinction: signatures prove authorization by a specific trusted signer. Anyone can compute a hash, but only the private key holder can produce a valid signature.

Measured Boot — detection

At each boot stage, a cryptographic hash is calculated and stored in TPM PCR registers. PCR 0 → UEFI firmware, PCR 4 → bootloader, PCR 8–9 → GRUB config and kernel. PCR registers can only be **extended** (never overwritten without rebooting). Creates an unforgeable record of everything that executed during boot. Secure Boot = prevention (blocks unauthorized code). Measured Boot = detection (records what ran for later verification via remote attestation). Both use both.

Topic 9: TPM 2.0 and Attestation

What a TPM is

A dedicated security chip soldered to the motherboard. Separate hardware from the CPU, with its own processor, secure storage, and firmware. A tamper-proof safe bolted to the server. Stores keys that never leave the chip and signs attestation statements.

PCR extend operation

$\text{new_PCR_value} = \text{hash}(\text{old_PCR_value} + \text{new_measurement})$. A one-way chain — can't work backward to fake steps, can't reset without rebooting.

Remote attestation

1. Attestation service challenges node: "Prove what you booted."
2. Node's TPM signs its PCR values with a key only that TPM holds.
3. Service compares signed PCR values against known-good expected values.
4. Match → trusted. Mismatch → quarantined.

Critical rule: Before a node gets any secrets (encryption keys, model weights, API credentials), it must pass attestation. Untrusted nodes are locked out.

Topic 10: Immutable OS Images

dm-verity — runtime tamper detection

Builds a hash tree (Merkle tree) of every block on disk. Every disk read is hash-checked in real time. A single modified byte causes a read failure. Key distinction: Secure Boot verifies at boot time only. dm-verity protects at runtime.

A/B image updates

Two OS partitions. Running system on A, new image written to B. Next reboot boots B. If B fails, automatic fallback to A. Updates are **atomic** — fully applied or not applied at all.

UKI (Unified Kernel Images)

Bundles kernel, initrd, and command line into a single signed file. One signature check instead of multiple. Eliminates attacks that swap individual components.

Topic 11: Hardened Ubuntu and CIS Benchmarks

- **CIS Level 2** — strict security configuration: disabled unused filesystems/protocols, strict file permissions, audit logging, minimal packages.
- **FIPS-validated kernels** — cryptographic implementations formally certified. Often required by sovereign customers.
- **AppArmor enforce mode** — confines each program to a limited set of resources. Blocks violations in real time.
- **Minimal package set** — no compiler, no package manager, no debugging tools in production.

How security layers compound

Secure Boot → Measured Boot + TPM → dm-verity → CIS hardening → FIPS kernel → AppArmor enforce. No single layer is unbreakable. An attacker must defeat all of them.

Phase 3 — The big picture

Boot integrity (Secure Boot + Measured Boot) → hardware-anchored proof (TPM + attestation) → tamper-proof filesystem (dm-verity + A/B updates) → minimal, locked-down OS (CIS + FIPS + AppArmor). A Brub node proves what it booted, can't be modified while running, and confines every process. Secrets only released to verified nodes.

Phase 4: Containers, Kubernetes, and Kata

Topic 12: Container Fundamentals

What a container actually is

Not a virtual machine. A regular Linux process with two isolation mechanisms:

- **Namespaces** — process sees its own version of system resources (PID, network, mount, user namespaces).
- **Cgroups** — limit resource consumption: CPU time, memory, disk I/O, network bandwidth.

Containers share the host kernel. Strength: no OS overhead, start in milliseconds. Weakness: kernel is shared attack surface — a kernel vulnerability lets any container escape to the host.

Important: A "Ubuntu container" doesn't run Ubuntu's OS. It packages Ubuntu's **userspace** (tools, libraries) but uses the host kernel. Docker images are filesystem snapshots, not bootable OSes.

Topic 13: Kata Containers Deep Dive

Container vs. VM — the fundamental difference

A container is isolated by **software rules the kernel enforces on itself**. A VM is isolated by **CPU hardware** (Intel VT-x, AMD-V) creating a separate execution environment. Kernel vulnerabilities break container isolation but not VM isolation.

Kata architecture

1. QEMU/KVM creates a lightweight microVM with a tiny guest kernel
2. kata-agent inside the microVM receives instructions from the host
3. Container workload runs inside the microVM
4. Host-to-VM communication uses vsock (direct channel, not network)

Kubernetes sees a container. Underneath: hardware-enforced VM boundary.

GPU passthrough with VFIO

VFIO maps a physical GPU's hardware registers and memory directly into the microVM's address space. Near-native performance with VM-level security.

Practical lab example

GPU 41:00.0 → NVIDIA driver (host/Docker testing). GPU 81:00.0 → VFIO-PCI driver (Kata passthrough via Kubernetes). Test pod uses `runtimeClassName: kata-qemu-nvidia-gpu` and requests `nvidia.com/pgpu: 1` (exclusive physical GPU passthrough). `uname -a` inside the pod proves isolation by showing the guest kernel.

Topic 14: Kubernetes Core Concepts

Core objects

- **Pod** — smallest deployable unit. In Bruk: single inference worker inside Kata microVM with passthrough GPUs.
- **Node** — a physical machine (bare-metal GPU server). Runs kubelet agent.
- **Control plane** — API server (front door), Scheduler (placement decisions), etcd (cluster state store). Runs on separate machines, not GPU nodes.

Scheduling in Bruk's model

Unlike shared cloud clusters (bin-packing), Bruk uses bare-metal, single-tenant scheduling: match pods to correct GPU type, respect NVLink affinity, ensure tenant workloads land only on their dedicated nodes via taints/tolerations.

Security controls

- **RBAC** — controls what humans and service accounts can do through the Kubernetes API. Locks tenants to their own namespace.
- **Pod Security Admission** — controls what pods themselves are allowed to do. Three levels: Privileged (system only), Baseline (prevents escalation), Restricted (tenant workloads — no root, no host mounts).
- **NetworkPolicies** — controls pod-to-pod traffic. Default-deny: every connection must be explicitly permitted. Enforced by Cilium via eBPF.

Two APIs — don't confuse them

- **Kubernetes API** — built-in, written in Go. Cluster operations (creating pods, applying policies). Only admins and internal components.
 - **Inference API** — user-facing endpoint for prompts/responses. vLLM exposes OpenAI-compatible API. Envoy proxy handles TLS, auth, rate limiting.
-

Topic 15: Cilium and eBPF Networking

What eBPF is

Lets you run small, sandboxed programs directly inside the Linux kernel without modifying it. The kernel verifies safety (no crashes, no infinite loops) then attaches the program to a hook point. Every event at that hook fires the eBPF program at kernel speed.

Why eBPF beats iptables

iptables walks a linear chain of rules — $O(n)$. eBPF does a hash map lookup — $O(1)$. A cluster with 10 policies and one with 10,000 policies have the same lookup speed.

How Cilium uses eBPF

Replaces kube-proxy entirely. Compiles NetworkPolicies into eBPF programs. Packet leaves a pod → eBPF fires → hash map lookup on source/destination identity → allow/deny in microseconds. No userspace involvement.

Identity-based policy

Traditional policies use IP addresses — but pods are ephemeral and IPs get recycled. Cilium assigns each pod a **cryptographic identity** based on Kubernetes labels. Policies reference identities, not IPs. Pod restarts with new IP but same labels → still works. Different pod inherits old IP with different labels → denied.

Default-deny east-west

East-west = traffic between pods inside the cluster. Default-deny means no pod can talk to any other pod unless a policy explicitly allows it. Enforced efficiently because eBPF lookups add negligible latency.

Tenant isolation is defense in depth

Kubernetes scheduling (taints/tolerations) → Kata + VFIO (hardware GPU isolation) → RBAC (API-level separation) → NetworkPolicies via Cilium (network isolation) → Pod Security Admission (privilege restriction). If any one layer fails, the others still hold.

Deep Dive: How Transformer Inference Works

Cuts across GPUs, KV-cache, RDMA, and disaggregation — central to understanding Bruk's design.

The attention mechanism

Every transformer layer computes three vectors per token from learned weight matrices:

- **Query (Q)** = token_embedding $\times W_q$ — "What am I looking for?"
- **Key (K)** = token_embedding $\times W_k$ — "What do I contain?"
- **Value (V)** = token_embedding $\times W_v$ — "What information do I carry?"

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \times K^T / \sqrt{d_k}) \times V$$

$Q \times K^T$ — relevance scores between all token pairs. Scaled by $\sqrt{d_k}$ to prevent softmax saturation. Softmax converts to probability distribution. Multiplied by V to get weighted combination of value vectors.

Multi-head attention

Done h times in parallel (32–128 heads). Each head has its own W_q, W_k, W_v , so each learns different patterns (syntactic, semantic, positional). Outputs concatenated and projected through W_o .

Full transformer layer

input $\rightarrow$ multi-head attention $\rightarrow$ add & normalize $\rightarrow$ FFN $\rightarrow$ add & normalize $\rightarrow$ output

FFN: two large matrix multiplications with activation function (SiLU, GeLU) between them. Often holds the majority of parameters. Activation functions run on CUDA cores; matrix multiplications on tensor cores. Stack 80–126 layers = a full LLM.

Prefill, KV-Cache, and Decode

Prefill phase

All input tokens processed through all layers simultaneously. Compute-heavy, GPU parallelism fully utilized. At each layer, K and V vectors for every token are computed and stored as the KV-cache.

KV-cache — what it stores

After prefill, the cache contains every layer's K and V vectors for every input token. Without caching, generating each new token would require recomputing all previous tokens' K and V from scratch — quadratic cost.

Decode phase

For each new token: compute Q, K, V for that one token only $\rightarrow$ attention against all cached keys and values $\rightarrow$ append new K, V to cache $\rightarrow$ predict next token $\rightarrow$ repeat.

Why decode is memory-bandwidth bound

For each new token at each layer: read W_q, W_k, W_v weight matrices (huge) plus entire KV-cache (grows with sequence). Compute: one small vector $\times$ matrix multiplication. Ratio: gigabytes read, tiny math done. GPU cores finish instantly, then wait for next data chunk from HBM.

KV-cache sizing

Llama 405B (126 layers, 128 heads, 128 head dimension, FP16): ~ 64 KB per token per layer $\rightarrow \sim 8$ MB per token across all layers $\rightarrow \sim 32$ GB for a 4096-token sequence. This is why PagedAttention manages cache like virtual memory pages and RDMA transfer between servers is a critical performance path.

	Prefill	Decode
Processes	All input tokens at once	One new token at a time
Bottleneck	Compute (lots of math)	Memory bandwidth (reading weights + cache)
GPU utilization	High (parallel work)	Low (small work, big reads)

The full inference flow

Prompt → [Prefill GPU] process all tokens, create KV-cache → [RDMA Write over Spectrum-X] move KV-cache to decode GPU → [Decode GPU] generate tokens one at a time, each streamed back to client. This asymmetry is why Bruk separates them onto different GPU pools (disaggregated inference).

Phase 4 (continued): Cilium, GitOps, GPU Operator

Topic 15: Cilium and eBPF Networking

The problem with iptables

Traditional NetworkPolicy enforcement uses iptables — a linear chain every packet walks through. $O(n)$ per packet. With thousands of rules in a large cluster, this becomes a bottleneck.

What eBPF is

eBPF lets you run sandboxed programs directly inside the Linux kernel. The kernel verifies safety then attaches the program to a hook point. For networking: direct hash map lookup instead of chain walking. **$O(1)$ instead of $O(n)$** . A cluster with 10,000 policies has the same lookup speed as one with 10.

How Cilium uses eBPF

Replaces kube-proxy entirely. Compiles NetworkPolicies into eBPF programs. Packet leaves pod → eBPF fires → hash map lookup on identity → allow/deny in microseconds. No userspace, no iptables.

Identity-based policy

Pods are ephemeral — IPs get recycled. Cilium assigns cryptographic identity based on Kubernetes labels, not IP. Pod restarts with new IP but same labels → still allowed. Different pod inherits old IP with different labels → denied.

Default-deny east-west

East-west = all pod-to-pod traffic cluster-wide (same node or different nodes). Default-deny: every connection must be explicitly permitted. A compromised pod in Tenant A can't probe Tenant B's pods anywhere in the cluster.

Topic 16: Helm, Flux, and GitOps

GitOps — the principle

The entire desired cluster state is declared in Git. Every manifest, chart value, and policy. Git is the single source of truth. No one runs kubectl apply against production directly. Ever.

- **Audit trail** — every change has a commit with timestamp, author, and diff. Verifiable answer to 'what ran on June 3rd at 14:00?'
- **Review process** — pull requests mean no change goes live without peer review
- **Rollback** — revert a commit, Flux applies it, cluster returns to previous state within minutes
- **Reproducibility** — entire cluster recreatable from the repo

Helm — packaging Kubernetes manifests

Package manager for Kubernetes. Charts are templated YAML with configurable values.yaml. Change replicas: 3 to replicas: 5 → Helm generates two additional pod specs automatically.

Flux — the reconciliation engine

Runs inside the cluster, continuously watches Git: poll for changes → compare desired to actual → apply differences. If someone manually edits the cluster, Flux detects drift and reverts it. Continuous enforcement, not a one-time deploy.

Topic 17: NVIDIA GPU Operator

The problem it solves

Each GPU node needs: NVIDIA kernel drivers, CUDA runtime, container toolkit, device plugin, MIG config, DCGM exporter. Manual setup is error-prone. Critically: drivers can't be baked into the immutable OS image because driver updates would require a full OS rebuild.

Drivers as containers — the key insight

Instead of installing NVIDIA drivers on the host OS (which would write to the dm-verity protected filesystem), the GPU Operator runs the driver inside a privileged container that loads the kernel module dynamically. OS image stays clean. Driver updates are container image updates — no OS rebuild required.

DaemonSets

	Deployment	DaemonSet
Says	Run N replicas somewhere	Run one on every matching node
Use case	Application workloads	Infrastructure agents
GPU Operator	Not used	One per component per GPU node

The GPU Operator creates one DaemonSet per component (driver, device plugin, DCGM exporter). When a new node joins, pods land on it automatically.

Full node onboarding — no human required

1. Node boots → passes attestation (Phase 3)
2. Flux reconciles it into the cluster (Topic 16)
3. GPU Operator detects new node
4. Automatically deploys all GPU software via DaemonSets
5. Node ready to accept inference workloads

Adding GPU capacity is a hardware operation, not a software operation.

Phase 4 Complete — The full stack

Containers → Kata microVMs (VFIO GPU passthrough) → Kubernetes (scheduling, RBAC, Pod Security) → Cilium eBPF (identity-based, default-deny, O(1)) → GitOps (Helm + Flux + Git) → GPU Operator (DaemonSets + driver containers). Defense in depth: if any one layer fails, the others still hold.

Phase 5: Identity, Secrets, and Zero Trust

How each service proves who it is, and how authorization decisions are made without trusting the network. Every service must cryptographically prove its identity on every request.

Topic 18: SPIFFE/SPIRE and Workload Identity

The problem with static credentials

Static API keys and passwords get leaked, don't expire, and say nothing about what is making the request. In a cluster with hundreds of services, managing them becomes a security nightmare.

SPIFFE — the standard

SPIFFE (Secure Production Identity Framework for Everyone) defines how workloads get cryptographic identities. The core concept is the **SVID (SPIFFE Verifiable Identity Document)** — a cryptographically signed document identifying a workload as a URI:

`spiffe://bruk.example.com/tenant-a/inference-worker`

- **X.509 SVID** — a TLS certificate with the SPIFFE ID in the Subject Alternative Name field. Used for internal service-to-service mTLS.
- **JWT SVID** — a signed JSON Web Token containing the SPIFFE ID. Used for external-facing calls.

SPIRE — the implementation

- **SPIRE Server** — runs centrally. The certificate authority. Knows the trust policy: which pod/service account/namespace maps to which SPIFFE ID. SVIDs are issued on demand, not stored.
- **SPIRE Agent** — runs on every node as a DaemonSet. Workloads contact it via Unix socket. Agent uses kernel-level attestation to verify the workload's identity, then requests the SVID from the server on its behalf.

The attestation flow

Workload starts → contacts SPIRE Agent via Unix socket → agent uses kernel-level attestation to verify pod/namespace/service account → agent requests SVID from SPIRE Server → server validates against policy → issues X.509 certificate → agent delivers SVID to workload → workload uses SVID for mTLS.

Short-lived certificates

SVIDs expire in hours, not years. A stolen certificate is useless within hours automatically — no manual revocation. This changes the attacker's calculus: credentials have a very short window of usefulness. SPIRE automatically renews before expiry — no human involvement.

mTLS (mutual TLS)

Normal TLS: client verifies server. mTLS: both sides verify each other's X.509 SVIDs. A compromised service without a valid SVID cannot impersonate a legitimate one. Each tenant's workloads get SPIFFE IDs scoped to their tenant — cryptographically distinct identities.

Topic 19: OPA (Open Policy Agent) and WASM

Identity vs. authorization

SPIFFE/SPIRE answers 'who are you?' OPA answers 'what are you allowed to do?' A valid SVID proves identity — but without OPA, every service implements its own authorization checks inconsistently. OPA centralizes that logic: one policy engine, one audit trail, one place to change rules.

What OPA is

A general-purpose policy engine. Policies are written in **Rego** — a declarative language that evaluates input data and produces allow/deny decisions. Policies live in Git, reviewed via pull requests, deployed centrally.

OPA compiled to WASM — in-process evaluation

OPA normally runs as a sidecar — every auth check is a network call, adding latency. OPA can compile Rego policies to a **WASM binary** embedded directly inside Envoy. Runs in-process — no network hop, no sidecar. Policy evaluation in microseconds inside the same process handling the request.

Policy as code

Authorization logic is version-controlled in Git. Changes go through pull request review. Every decision is logged with input, matching policy, and outcome. A customer can ask 'what policy governed access to my models on June 3rd?' — you can show the exact Git commit, who approved it, and the diff.

What OPA evaluates in Bruk:

- Request authorization (does this tenant have permission to use this model?)
- Rate limiting (has this tenant exceeded their quota?)
- PII validation (does this request contain data that shouldn't be logged?)
- Kubernetes admission control (should this pod be allowed to run?)

Topic 20: Envoy Proxy Deep Dive

What Envoy is

A high-performance proxy (written in C++) that sits in front of every service in Bruk. Every request passes through Envoy before reaching its destination. It is the enforcement layer where identity and policy are applied to live traffic.

What Envoy handles

- **TLS termination and mTLS** — terminates inbound TLS, validates X.509 SVIDs, initiates mTLS upstream. Applications never handle TLS. Fix a TLS bug in Envoy once, not in 50 services.
- **JWT validation** — validates signature, checks expiry, extracts claims. Invalid JWTs rejected before touching the inference stack.
- **Rate limiting** — per-tenant request rate limits. Exceeding limit returns 429 before consuming any GPU resources.
- **PII redaction** — scrubs sensitive fields from requests before logging. Inference worker processes the full request; logs are clean.
- **xDS routing** — dynamic routing via discovery service API. Control plane (Dynamo) pushes routing decisions in real time, no restart required.

External vs. internal identity

	External clients	Internal services
Mechanism	JWT in Authorization header	X.509 SVID / mTLS
Issued by	Auth service	SPIRE
Verification	Signature + expiry check	Mutual certificate validation
Direction	One-way (client proves identity)	Mutual (both sides verify)

xDS vs. traditional load balancers

Traditional load balancer: static config file, edit and reload to change routes. Envoy with xDS: no static routing — the control plane (Dynamo) pushes routing decisions over a live API connection instantly. Enables per-request routing based on real-time state: 'send this request to worker 7 because it already has the KV-cache for this

conversation.'

The enforcement chain on every request

TLS termination + SVID/JWT validation (identity) → OPA WASM evaluation (authorization) → rate limit check (resource protection) → PII redaction (data protection) → xDS routing (intelligent placement) → request reaches inference worker.

If any step fails, the request is rejected before reaching the GPU. The inference stack only ever sees authenticated, authorized, rate-limited, clean requests.

Phase 5 — The big picture

SPIFFE/SPIRE (every service gets a cryptographic identity) → OPA in WASM (every request evaluated against policy in microseconds, in-process) → Envoy (the enforcement point). Every request is authenticated, authorized, rate-limited, and routed before it ever touches a GPU. This is what 'replace trust with verify' looks like in practice.